TARGETX — TECH STACK & PROGRAMMING LANGUAGES

Comprehensive Technical Architecture, Domains, Sub-Domains & Framework Specification

Prepared for Chief Technology Officers, Software Engineering Leads & Academic Tech Jurors

1. TECHNOLOGY MATRIX AT A GLANCE

TargetX was engineered using an enterprise polyglot micro-architecture combining high-performance scientific Python computing with a full-stack, end-to-end type-safe TypeScript/Node.js web platform:

Language	Core Domain	Key Sub-Domains	Primary Libraries & Engines
Python 3.10+	AI, ML & Biophysics	Deep Learning, GBDT, XAI, Biophysics	PyTorch, XGBoost, LightGBM, SHAP,
TypeScript 5.9+	Full-Stack Type System	Type-Safe RPC, Contracts, Schemas,	tRPC v11, Zod v4, Drizzle ORM, Node
JavaScript / Node	Async Server Runtime	Event Loop, REST Routing, Reactive UI	Node.js 20+, Express 4, React 19,
CSS3 / HTML5	Bio-Informatics UI/UX	Glassmorphism, SVG Visuals, Tokens	Tailwind CSS v4, Recharts, Framer
SQL (Relational)	Data Persistence	Relational Schemas, Migrations, ACID	MySQL / SQLite via Drizzle ORM
Dockerfile / Shell	DevOps & Deployment	Multi-Stage Builds, Bundling, PaaS	Docker, Vite 7, esbuild, Render Cloud

2. PYTHON (V3.10+) — AI, DEEP LEARNING & BIOPHYSICS

Python

Version: 3.10.x / 3.11.x **Domain: Scientific Computing & Machine Learning**

- **Deep Learning & Attention Networks:** PyTorch (torch, torch.nn) powering the DeepHF Conv-Attention model (1D-CNN with kernel sizes 3 & 5 + MultiheadAttention num_heads=4 + Batch Normalization) and DeepSpCas9 CNN for sequence-dependent cutting kinetics.
- **Gradient Boosted Decision Trees (GBDT):** XGBoost (xgboost) and LightGBM (lightgbm) trained on 30-dimensional engineered genomic feature vectors, capturing non-linear nucleotide interactions and GC-skew dynamics.
- **Bagged Ensembles & Stacking Meta-Learner:** Scikit-Learn (ExtraTreesRegressor, RandomForestRegressor, Ridge) forming the meta-stacking ensemble layer that yields Pearson r = 0.9782 and RMSE = 0.0299.
- **Biophysical Thermodynamics & Repair:** Nearest-neighbor thermodynamic algorithms implementing the SantaLucia thermodynamic free energy (Delta G, Delta H, Delta S) and inDelphi microhomology (MMEJ vs NHEJ) repair outcome predictors.
- **Explainable AI (XAI):** SHAP (shap.TreeExplainer) for game-theoretic feature attribution, breaking down exact delta contributions per nucleotide position for laboratory validation.
- **Data Serialization & Benchmarking:** NumPy and Pandas for genomic vector transformation; Joblib for high-throughput model weight serialization and loading.

3. TYPESCRIPT (V5.9+) — TYPE SAFETY & DOMAIN ARCHITECTURE

TypeScript

Version: 5.9.3 (Strict Mode) **Domain: Full-Stack Type-Safe System & Contracts**

- **End-to-End Type Safety (tRPC v11):** Uses @trpc/server, @trpc/client, and @trpc/react-query to share compile-time data contracts between server and client without code generation or manual OpenAPI schemas.

- **Biological Input Validation (Zod v4):** Validates user prompts, IUPAC nucleotide sequences (A, C, G, T, N), genomic loci coordinates, and organism identifiers at the API boundaries to prevent invalid bioinformatics inputs.
- **Object-Relational Mapping (Drizzle ORM):** Type-safe SQL query builder and schema management (drizzle-orm, drizzle-kit) defining models for Projects, Pipeline Stages, gRNA Candidates, Off-Target Loci, and SHAP results.
- **Asynchronous Pipeline State Machine:** Strictly-typed 8-stage state handler (server/pipeline/stageHandlers.ts) managing asynchronous transitions, error handling, telemetry stats, and database transaction tracking.

4. JAVASCRIPT (ES2022+) & NODE.JS — RUNTIME & REACTIVE UI

JavaScript / Node.js

Version: Node v20+ / ES2022 **Domain: Asynchronous Runtime & Frontend Engine**

- **Backend Application Server:** Node.js asynchronous event-driven I/O engine running Express 4 (express) for HTTP routing, CORS security headers, reverse-proxying, and session handling.
- **Frontend Reactive UI Library:** React 19 (react, react-dom) utilizing functional components, concurrent rendering, and context-based state management (ProjectContext).
- **Asynchronous Query & Caching:** TanStack React Query v5 (@tanstack/react-query) for intelligent query caching, background data revalidation, and live polling of pipeline stage telemetry.
- **Headless & In-Browser PDF Generation:** jsPDF (jspdf) engine enabling vector rendering of clinical research reports, jury guides, and executive briefing dossiers directly from application state.

5. CSS3 & HTML5 — STYLING, TOKENS & SCIENTIFIC VISUALS

Tailwind CSS & SVG

Version: Tailwind v4 / HTML5 **Domain: Bioinformatics UX & Visualization**

- **Design System & Glassmorphism:** Tailwind CSS v4 with custom utility tokens, dark mode backdrop filters (backdrop-blur-md, bg-[#0d1117]/85), and bioluminescent neon cyber-cyan borders.
- **Interactive Scientific Visualizations:** Recharts (recharts) rendering responsive SVG charts for model radar scores, GC-content curves, nucleotide distributions, and variance metrics.
- **Micro-Animations & HUD Steppers:** Framer Motion (framer-motion) driving fluid UI animations across the 8-stage Genomic HUD Stepper, glowing laser connectors, and real-time telemetry tickers.
- **Accessible Design Primitives:** Radix UI primitives (@radix-ui/react-*) ensuring WAI-ARIA accessibility across modal dialogs, tabs, tooltips, dropdowns, and sliders.

6. DEVOPS, CONTAINERIZATION & SQL DATABASE

- **Multi-Stage Docker Container:** Containerized via multi-stage Dockerfile running Node.js alpine base images, caching dependencies, and isolating production artifacts for cloud execution.
- **Production Bundling & Compilers:** Vite 7 (vite) for rapid frontend tree-shaking and HMR; esbuild (0.25+) for sub-second Node.js TypeScript server bundling into an optimized ESM dist file.
- **Cloud Platform-as-a-Service (Render PaaS):** Live continuous deployment on Render Cloud PaaS (https://target-x.onrender.com) with automated environment variable injection, TLS termination, and health checks.
- **Relational Persistence (SQL):** Structured schemas supporting ACID transactions across multi-step pipeline results, candidate rankings, and off-target mismatch matrices.

7. ARCHITECTURAL JUSTIFICATION (WHY THIS POLYGLOT STACK?)

Why not build everything purely in Python or purely in Node.js? Tech jurors often look for architectural justification:

- **Separation of Concerns:** Python is the undisputed global benchmark for tensor operations (PyTorch), machine learning (XGBoost), and bioinformatics algorithms. However, Python web frameworks (Flask/Django) have slower concurrency and lack native modern reactive UI ecosystems.
- **Node.js + React Concurrency:** TypeScript/React provides ultra-fast event loop concurrency, reactive state management, and instant HMR, giving users a sleek, commercial-grade biotechnology HUD command center.
- **End-to-End Type Safety:** tRPC and Zod eliminate runtime schema mismatches between the front-end telemetry and backend pipeline, making the system resilient to unexpected bioinformatics payload errors.